are you familiar with slr(1) parsing with epsilon rules and moves of an slr(1) parser .? Uploaded files: - Moves of an SLR(1) parser on id _ id.md (63 KB)

Yep, familiar with both — SLR(1) parsing (shift-reduce with the ACTION/GOTO tables built from LR(0) items + FOLLOW sets), and how it handles ε-productions (a reduce action fires on the lookahead directly, without a shift, since there's nothing on the input to consume for the empty side).

I looked at your uploaded file — it's a step-by-step trace of an SLR(1) parser processing **id * id** on the classic grammar:

1. E -> E + T
2. E -> T
3. T -> T * F
4. T -> F
5. F -> (E)
6. F -> id

It walks through each move (shift id, reduce F→id, reduce T→F, shift , *shift id, reduce F→id, reduce T→T*F, reduce E→T, accept), using the ACTION/GOTO table you've got at the top, with annotations explaining what a shift action and a reduce action do to the stack.

This grammar has no ε-productions though — every production has at least one symbol on the right. If you want, I can:

1. Walk you through this exact trace move-by-move so you fully understand it, or
2. Explain how SLR(1) parsing changes when the grammar **does** have ε-productions (e.g., how the item sets, FOLLOW-based reduce actions, and empty reductions work), possibly with a different example grammar

Which would help more right now?

I'll give you to solve some problems…. but make sure always in your answer you will use state transition diagram like this for the whole LR(0) automation : a + ————-+ | | +————+ | | I0 |—+ | S'→•S | | S→•AA |—-A—-> I1 | A→•aA | | A→•b |—-b—-> I2 +————+ then first and follow computaton like this : First(V) = {int , float} Follow (v) = {$} then SLR(1) proper parse table which including

colums state , actions , goto like in the .md file and if asked for moves do it like in .md file

Got it — for every problem you give me, I'll follow this exact structure:

1. **LR(0) automaton** as a state transition diagram (boxes for each item set I0, I1, I2…, with kernel/closure items listed inside, and labeled arrows showing transitions on grammar symbols)
2. **FIRST and FOLLOW sets** in the compact format: First(X) = {...} Follow(X) = {...}
3. **SLR(1) parsing table** with State | Action (shift/reduce/accept) | GoTo columns, same layout as your uploaded file
4. **Moves of the parser** (if asked) as a Stack | Symbol | Input | Action trace, same as your .md file

Go ahead and send me the grammar(s) / problem(s) you'd like solved, and I'll work through each one in that format.

V–>PQ P–>int p–>float Q–>ϵ Q–>[size]Q also make sure you use augmentation first

Great — here's the grammar first, augmented as required:

0. V' -> V
1. V -> P Q
2. P -> int
3. P -> float
4. Q -> ε
5. Q -> [size] Q

LR(0) Canonical Collection (Item Sets)

I0:
V'-> .V
V -> .P Q
P -> .int
P -> .float

I1:
V'-> V. (Accept)

I2:
V -> P.Q
Q -> . <-- (Q -> ε, dot at end)
Q -> .[size] Q

I3:
P -> int.

I4:
P -> float.

I5:
V -> P Q.

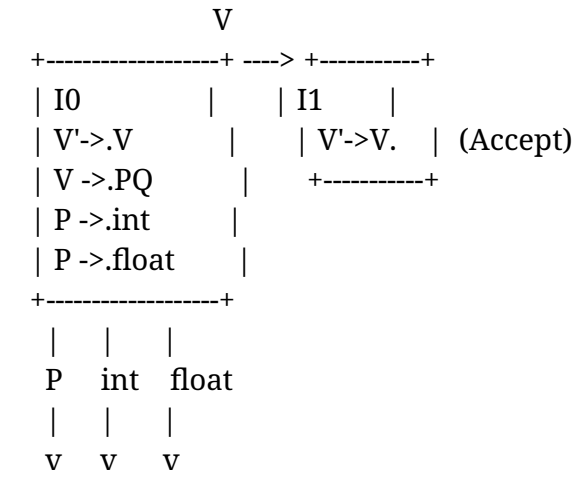
I6:
Q -> [.size] Q

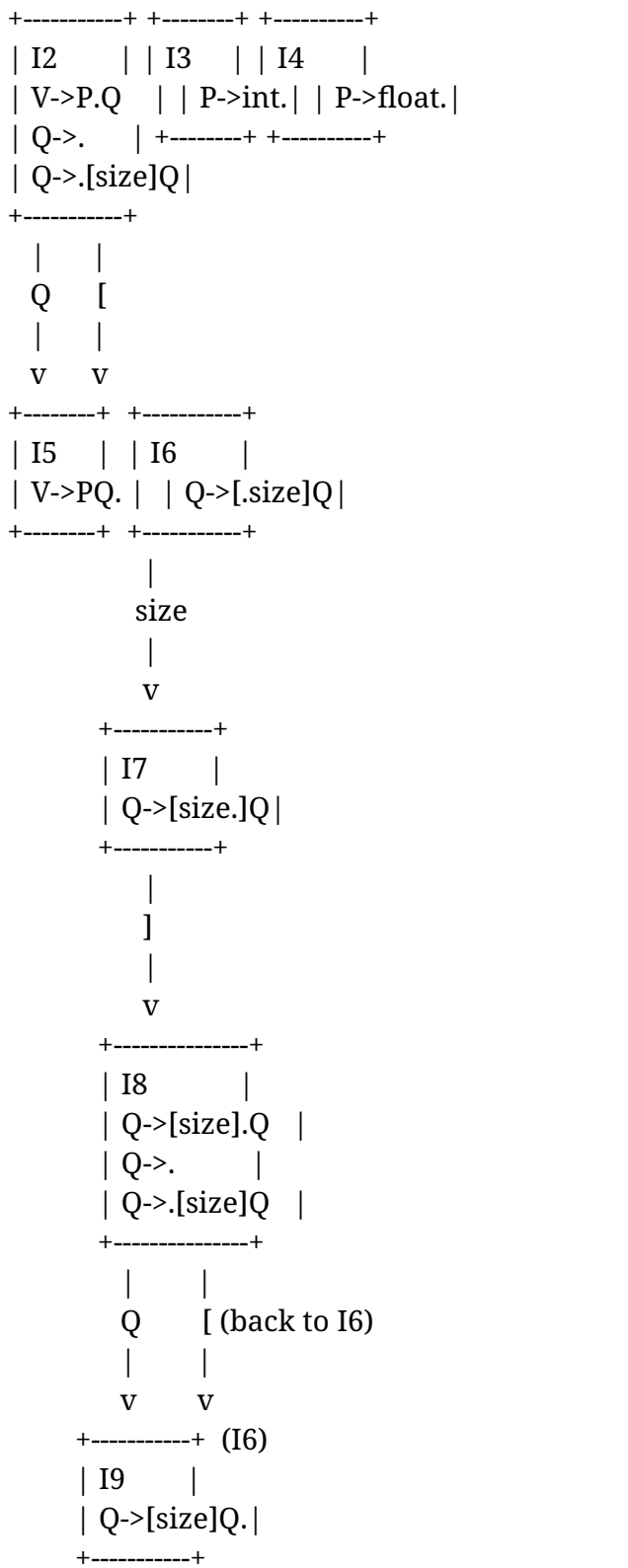
I7:
Q -> [size.] Q

I8:
Q -> [size].Q
Q -> . <-- (Q -> ε, dot at end)
Q -> .[size] Q

I9:
Q -> [size] Q.

State Transition Diagram




I2
V->P.Q
Q->.
Q->.[size]Q
I3
P->int.
I4
P->float.
Q
[
I5
V->PQ.
I6
Q->[.size]Q
size
I7
Q->[size.]Q
]
I8
Q->[size].Q
Q->.
Q->.[size]Q
Q
[(back to I6)
(I6)
I9
Q->[size]Q.

FIRST and FOLLOW

First(V) = {int, float}
First(P) = {int, float}
First(Q) = {[, ε}

Follow(V) = {$}
Follow(P) = {[, $}
Follow(Q) = {$}

SLR(1) Parsing Table

State	int	float	[	size	]	$	V	P	Q
0	S3	S4					1	2	
1						Accept			
2			S6			R4			5
3			R2			R2			
4			R3			R3			
5						R1			
6				S7					
7					S8				
8			S6			R4			9
9						R5			

Reduce key:

- R1 = reduce by V -> P Q
- R2 = reduce by P -> int
- R3 = reduce by P -> float
- R4 = reduce by Q -> ε
- R5 = reduce by Q -> [size] Q

Send me an input string (e.g. int [size] $ or float $) whenever you want the **moves of the parser** traced in the Stack | Symbol | Input | Action format.